

YouTube Documentation Guide

Dual-MCU IoT Sensor Node Project

Channel Positioning · Video Structure · Per-Phase Video Plan · Titles · Descriptions · What to Record

The Core Principle

The internet is full of embedded systems tutorials that show you the happy path: write the code, it works, here's the output. That content is abundant and forgettable. What is scarce — and what actually builds an audience — is content that shows **the engineering process**: the decision, the failure, the diagnosis, the fix.

Your channel is not a tutorial channel. It is an **engineering journal**. The difference is: a tutorial teaches someone to reproduce a known outcome. A journal documents a real engineering process, including what went wrong. The second is rarer, more trusted, and more useful to the exact audience you want: other embedded systems learners and hiring engineers.

Channel Positioning

You are: an intermediate programmer building a serious embedded systems project from scratch, documenting every step honestly. You are not a guru. You don't have all the answers at the start of each video. That is your differentiator.

Don't be this	Be this instead
"Tutorial: How to blink an LED on STM32"	"Phase 0 — Setting Up a Real Embedded Dev Environment on Fedora Linux (No IDE, No Arduino)"
"I2C Made Easy in 5 Minutes"	"I2C Took Me 3 Days — Here's What the Reference Manual Doesn't Tell You"
"FreeRTOS Task Creation Tutorial"	"Porting FreeRTOS to STM32 by Hand: What CubeMX Hides From You"
"How to do OTA updates"	"How I Wrote a Custom STM32 Bootloader From Scratch — And What Broke First"

Video Structure Template

Every video in this series follows this structure. Consistency builds audience trust and makes editing faster.

00:00 — Cold Open (30–60 seconds)

Start with the end result or the problem, not the introduction. Examples: the OLED displaying live sensor data, the OTA update completing, the HardFault screen print that diagnosed your bug. Then cut to: 'Here's how we got here.'

■ Never start with 'Hello everyone, welcome back to my channel.' That's the fastest way to lose a viewer in the first 5 seconds.

00:30 — What We're Doing (60–90 seconds)

One paragraph, no bullet lists, spoken naturally. What phase of the project is this. What specifically is being built today. What the viewer will learn or see. One sentence on prerequisites.

02:00 — The Build / The Work (8–15 minutes)

This is the core. Show the actual work: reading the datasheet, writing the code, running into errors, diagnosing them, fixing them. Do not edit out the failures — they are the most valuable content. Show your terminal, your UART output, your register dumps. Think out loud. Explain what you're reading in the RM and why.

■ The single most common mistake: over-editing. Don't try to make it look like you knew exactly what to do all along. Authentic confusion followed by genuine understanding is more valuable than a polished tutorial.

14:00 — It Works / Results (2–3 minutes)

Show the working system. UART output, OLED display, MQTT dashboard, whatever the deliverable is. Be honest about what works and what doesn't yet. If something still has a bug — say so. What's the next video?

16:00 — What I Learned (1–2 minutes)

The single most important section for long-term channel value. One or two genuine insights from this build. Not 'I learned a lot' — specific technical insights: 'The STM32F4 I2C peripheral requires a bus recovery sequence after any NACK because it cannot recover autonomously.' This is the content people bookmark and share.

18:00 — Outro (30 seconds)

Next video in the series. GitHub link. That's it. No 'smash that like button' — it's cringe and your audience knows it.

Per-Phase Video Plan

These are concrete video briefs. Each has a title formula, what to show, what the 'What I Learned' section should cover, and what makes this video worth watching over other content.

V01

"Setting Up a Real STM32 Dev Environment on Fedora Linux — ARM GCC, OpenOCD, No IDE"

What to show:

- Show terminal: dnf install commands, verify arm-none-eabi-gcc --version
- Show Makefile from scratch: what each flag means (-mcpu, -mfloat-abi, etc.)
- Show OpenOCD connecting to Black Pill via ST-Link clone: first successful flash
- Show directory structure being created and first commit pushed to GitHub

Key pitfall to demonstrate: The STM32 Black Pill has no onboard ST-Link (unlike Nucleo) and requires a udev rule. Show the exact error you get without the udev rule and the fix.

Why this video is worth watching: Every embedded beginner has this toolchain setup experience. This video is the one they search for when IDE-based tutorials don't work on their machine.

V02

"Writing a UART Driver From Scratch on STM32 — Register Level, No HAL, No Libraries"

What to show:

- Open RM0383 on screen. Show how to navigate to USART chapter.
- Walk through the init register sequence — each line with its RM citation.
- Show polling TX working: 'UART initialised' on minicom/screen.
- Show the ring buffer code and explain the head/tail masking trick.
- Show printf retargeting: _write() override, test with a format string.

Key pitfall to demonstrate: The most common beginner mistake: forgetting to enable the peripheral clock in RCC before touching any USART register. Show the symptom (nothing happens) and the fix.

Why this video is worth watching: This is the video that gets bookmarked. Register-level UART with ring buffer is searched constantly and most results use HAL.

V03

"I2C Bus Deep Dive: STM32 Register Level + BMP280 Real Sensor Data"

What to show:

- Show the I2C SR1/SR2 state machine from RM0383.
- Show the first I2C transaction: read BMP280 chip ID register (0xD0 → expect 0x60).
- Show the BMP280 compensation formula implementation.
- Show live temperature and pressure on UART output.
- Show the I2C bus lockup — trigger it deliberately and show the stuck SDA.

Key pitfall to demonstrate: The STM32F4 I2C errata. Every STM32 I2C user eventually hits bus lockup. Show the symptom, explain the errata, implement and demonstrate bus recovery.

Why this video is worth watching: One of the most searched STM32 topics. The bus recovery section alone makes this video valuable.

V04

"HC-SR04 Ultrasonic Sensor: Why You Need a Level Shifter and Timer Input Capture"

What to show:

- Show the 5V vs 3.3V issue: oscilloscope/multimeter reading on echo pin.
- Show the level shifter wiring.
- Show Timer Input Capture setup from register level.
- Show distance calculation formula and live output.

Key pitfall to demonstrate: Most HC-SR04 tutorials ignore the voltage level issue. Showing it explicitly and using hardware Timer IC instead of busy-wait is unique value.

V05

"DHT22 Single-Wire Protocol: Implementing Microsecond Timing Without Busy-Wait"

What to show:

- Show the DHT22 timing diagram from the datasheet.
- Show why busy-wait breaks under FreeRTOS (preview of Phase 2).
- Implement with Timer Input Capture.
- Show live humidity + temperature output.

Key pitfall to demonstrate: The specific pitfall: DHT22 timing must survive RTOS context switches. This is almost never addressed in DHT22 tutorials.

V06

"Porting FreeRTOS to STM32 by Hand — No CubeMX, No Code Generation"

What to show:

- Show git submodule add for FreeRTOS-Kernel.
- Show which files to add to the Makefile and why.
- Walk through FreeRTOSConfig.h settings with explanations.
- Show first task creation and verify via UART output.
- Show the task architecture diagram for the sensor node.

Key pitfall to demonstrate: Priority inversion — explain what it is, why binary semaphore doesn't fix it, why mutex does. This concept trips up beginners constantly.

Why this video is worth watching: CubeMX-generated FreeRTOS is mysterious. This video demystifies it completely.

V07

"ESP32 ESP-IDF From Scratch: WiFi + MQTT — Why I Don't Use Arduino Framework"

What to show:

- Show ESP-IDF install on Fedora.

- Show WiFi station connection with event loop explained.
- Show MQTT publish to hivemq.com broker, verify on MQTT Explorer.
- Explain why ESP-IDF event loop vs Arduino's loop() matters for OTA.

Key pitfall to demonstrate: The ESP-IDF event loop architecture is unfamiliar to Arduino users. Spend 3 minutes on this — it's the key mental model shift.

V08

"Designing a Binary UART Protocol From Scratch — Frame Format, CRC, State Machine"

What to show:

- Show protocol_spec.md being written live.
- Explain frame format design decisions (SOF/EOF choice, byte stuffing need).
- Show CRC-16 implementation and test vectors on PC.
- Show parser state machine code.
- Demo: STM32 sends BMP280 data, ESP32 decodes it, appears on MQTT.

Key pitfall to demonstrate: The byte stuffing problem — if your payload can contain the SOF byte, you get false frame starts. Show this bug happening and the fix.

Why this video is worth watching: Custom binary protocol design from scratch has almost no good content online. This video fills a genuine gap.

V09

"Writing an STM32 Custom Bootloader From Scratch — Flash Layout, Jump Sequence, Linker Scripts"

What to show:

- Show STM32F411 flash sector map from datasheet.
- Show both linker scripts (bootloader.ld vs application.ld).
- Walk through the jump sequence code line by line.
- Show flash erase and write register sequence.
- Demo: bootloader jumping to application — LED blinks at target frequency.

Key pitfall to demonstrate: The VTOR relocation step. Without SCB->VTOR = app_addr, all interrupts in the application vector table point to the wrong addresses. Show the crash, add the line, show it working.

Why this video is worth watching: Custom STM32 bootloader content is rare. This is the most technically distinctive video in the series.

V10

"OTA Firmware Updates Over WiFi via MQTT — STM32 + ESP32 End-to-End"

What to show:

- Show the full OTA data flow diagram.
- Show ota_push.py running from laptop terminal.
- Show UART log on STM32 as it receives chunks and writes to flash.
- Show system rebooting into new firmware — the LED blink rate changes.
- Show what happens when you cut power mid-transfer (safety test).

Key pitfall to demonstrate: The power-loss safety test. Deliberately power-cycle during OTA and show that the bootloader detects the partial write and does not jump to corrupt code. This is the proof that your design is production-safe.

Why this video is worth watching: The complete end-to-end OTA demo is the payoff video of the series. It's the one that validates everything built before it.

V11

"72-Hour Soak Test: What Failed and How I Fixed It"

What to show:

- Show the system health MQTT dashboard with uptime, heap, watchdog count.
- Show the WiFi reconnection handling.
- Show whatever actually failed during the soak (be honest — something will).
- Show the watchdog triggering and the system recovering.

Key pitfall to demonstrate: Whatever actually failed during your soak test. Don't fabricate a clean result. The real failure is the most valuable content.

V12

"Designing My First IoT PCB in KiCad — From Dev Board to Custom Hardware"

What to show:

- Show the schematic side-by-side with the dev board wiring.
- Walk through key schematic decisions: decoupling caps, BOOT0 pull-down, SWD header, I2C pull-ups.
- Time-lapse or sped-up PCB layout.
- Show the Gerber preview before ordering.
- Show the bare PCB when it arrives.

Why this video is worth watching: The translation from breadboard prototype to PCB is a milestone most embedded beginners never reach. Documenting it makes this channel exceptional.

V13

"PCB Bringup — First Power On, What Failed, What Worked"

What to show:

- Show powering through USB with a current meter (cheap USB inline meter).
- Show flashing via SWD on the new PCB.
- Show sensor data appearing on MQTT from custom hardware.
- Be honest about what required a rework — solder bridges, wrong footprint, missing pull-up. These make the best content.

Key pitfall to demonstrate: Whatever required rework. Every first PCB has something. Show it without embarrassment — it's engineering, not a failure.

Why this video is worth watching: This is the video that closes the series. It shows the complete arc from breadboard to production hardware.

Practical Production Advice

Equipment

- **Camera:** Your phone. A modern phone in 1080p/30fps is more than adequate. Prop it up to look at your desk at a 45-degree angle so viewers see your screen and your hands simultaneously. Stability matters more than resolution.
- **Screen recording:** OBS Studio (free, open-source, available on Fedora via `sudo dnf install obs-studio`). Record your desktop while you work. This is 80% of the video content.
- **Microphone:** Any USB microphone or even a phone headset. Audio quality matters more than video quality. Test it with headphones. If you can hear the fan noise, get closer to the mic.
- **Editing:** DaVinci Resolve (free version available for Linux). You need: cut, speed ramp for boring compilation sections, text overlays for register names. Nothing else. Do not spend more than 2 hours editing a video — it's an engineering journal, not a film production.

Title Formula

The title must answer: who is this for, what will they see, and why is it different from what already exists. The formula that works for technical content:

```
Pattern: [What you built] + [How] + [The honest differentiator]

Examples:
STM32 UART Driver From Scratch – Register Level, No HAL
Custom STM32 Bootloader: Flash Layout, Jump Sequence, Linker Scripts
OTA Firmware Over MQTT: STM32 + ESP32 End-to-End
I2C Bus Lockup: The STM32F4 Errata Nobody Talks About
```

Description Template

```
One paragraph: what this video covers. Specific. No fluff.

Chapters (YouTube timestamp format):
00:00 - Cold open: [result/demo]
00:30 - What we're building
02:00 - [Main section 1]
07:00 - [Main section 2 / debugging]
13:00 - It works
15:30 - What I learned
17:30 - Next video

Hardware used: STM32F411 Black Pill, ESP32 DevKit V1, ST-Link V2
Source code: github.com/[your-repo]/tree/[tag for this video]

Resources mentioned:
- RM0383: [link to ST page]
- [Specific datasheet or doc referenced in the video]
```

The GitHub–YouTube Link

Every video maps to exactly one GitHub tag. The video description links to that tag. The tag's annotation links to the video. Someone reading the GitHub repo can watch the video for any milestone. Someone watching the video can read the code for that exact state of the project. This bidirectional link is what makes the overall documentation exceptional rather than merely adequate.

What Not to Do

- **Don't wait until it's perfect.** Record Phase 0 even if the toolchain setup took you 3 hours and was messy. The messiness is the content.
- **Don't re-record when you make mistakes.** Edit them out only if they're more than 2 minutes of confusion with no insight. Otherwise, leave them.
- **Don't over-script.** Know your three main points before recording. Speak naturally from those. A script makes technical content feel like a presentation.
- **Don't post inconsistently.** One video per phase is the target — roughly one per month. The audience for technical content is patient; they prefer depth over frequency.
- **Don't neglect the 'What I Learned' section.** It is the section most watched and most shared. It is also the section that proves you genuinely understand what you built — which is the whole point of this project.